

FINAL EXAMINATION

Course: CS202 – PROGRAMMING SYSTEMS	
Time: 100 minutes	Term: 1 – Academic year: 2023-2024
Lecturer(s): Dr. Dinh Ba Tien	
Student name:	Student ID:

(Notes: Closed Book Exam)

- Problem 1.** Give a **short answer** with an **example** for each of the following questions (2.Opts)
- What are the **access control specifiers** (i.e. visibility) of the members in a class? Briefly explain the differences among them.
 - What are the main differences between a **copy constructor** and a **copy assignment operator**?
 - What is the difference between a **pure virtual function** and a **virtual function**?
 - What are the advantages of **exception handling** using try-throw-catch in C++ comparing to the other old-styled exception handlings in C, e.g. terminating the program?

Problem 2. What would be printed on the screen if we run the following program? (3.0 pts)

<pre>#include <iostream> using namespace std; class B { public: B() {cout << "B::B()\n";} B(int t) {cout << "B::B(int)\n";} B(const B& src){cout<<"B::CopyCon\n";} B& operator=(const B& src) { cout << "B::operator=()\n"; return *this; } void print() { cout << "B::print()\n"; } virtual void doSth() { cout << "B::doSth()\n"; } virtual ~B() { cout << "B::~~B()\n"; } };</pre>	<pre>class D: public B { public: D() {cout<<"D::D()\n";} D(int t): B(t) {cout<<"D::D(int)\n";} D(int t, int k){ cout<<"D::D(int, int)\n";} D(const D& src):B(src) { cout<<"D::CopyCon\n"; } D& operator=(const D& src) { B::operator=(src); cout<<"D::operator=()\n"; return *this; } void print() { cout << "D::print()\n"; } void doSth() { cout << "D::doSth()\n"; } ~D() {cout << "D::~~D()\n";} };</pre>
--	---

```
void testing(D d, B* pB) {
    D x=d;
    x.print();
    x.doSth();
    cout << "Doing 3: " << endl;
    pB->print();
    pB->doSth();
}
```

```
int main() {
    cout << "Doing 1: " << endl;
    D x(2), y(5, 3), z;
    z = x;
    cout << "Doing 2: " << endl;
    testing(x, &y);
    cout << "Doing 4: " << endl;
    return 0;
}
```

Problem 3. Design and create a **template class** named **Container** that can store elements of any type. The class has a method named **print** that prints the elements and a method named **sort** which sorts the elements in an ascending order. We need to use **Container** to manage elements of a class called **Animal**. Each **Animal** instance has a **name** and a **weight**. **Weight** is used as the metric in comparisons between **Animal** instances.

Write a **main** function which provides an example of how you would use the **Container** class with **Animal**.

Problem 4. You are asked to design a system for a shop selling *Matryoshka dolls* (also known as stacking dolls or nesting dolls), one of the iconic toys of Russia. Each Matryoshka doll consists of two essential components: the outer shell (representing the doll itself) and the inner dolls nested within (see Figure 1 below). The inner-most doll does not nest anything. The appearance of the outer shell and the inner dolls typically look almost the same and are just different in their sizes. Currently, the shop sells three models of Matryoshka dolls, a **Japanese geisha wearing a kimono**, a **Russian princess in a sarafan** and a **Vietnamese girl in Ao Dai**. These three models have different design descriptions and prices. The price of a doll model is fundamentally based on the outer shell and the number of dolls nested within. The price of the outer shell varies across the models depending on their decorations. In this case, the outer shell doll's price can be manually set by the shop. The price of an inner-layer doll is equal to 80% of the price of the outer-layer doll which directly nests it (e.g. a doll with a nesting level of 1 will be priced at 80% the price of the doll at the nesting level 0 (the outer shell) and so on)



Figure 1. An illustrative image of a Matryoshka doll where all nested dolls are taken out of the outer shell to depict their appearance similarity and size differences.

For easy management of the dolls, each doll has an identity in the system. Besides that, the outer shell or inner dolls of each doll will be marked with a nesting level, starting from 0 for the outer shell and increased by 1 for each nesting layer. You are asked to do the followings:

- Provide a UML diagram and declarations of all classes (including attributes and signatures of methods) that you believe will be needed to implement the management system for this shop. Please consider how to design the system so that it will be easily maintainable and extendable in the future.
- Implement a main function to demonstrate how you would use all the classes declared above.
- Write source code to calculate the total price of all the dolls in the shop as well as the total price of the dolls of each model. Feel free to modify the classes with all necessary attributes and methods in order to perform the calculations.

-- GOOD LUCK --